

Lab 10: Spring WebFlux & WebClient

นายพีรพล แก้วเจริญสันติสุข 673380287-8 Section 1

1. Reactive Programming vs Blocking

Blocking (แบบเดิม): 1 request ใช้ 1 thread และ thread จะหยุดรอจนกว่างาน I/O (DB, HTTP) จะเสร็จ ถ้ามี request พร้อมกันมากต้องใช้ thread จำนวนมาก สิ้นเปลือง memory และ scale ยาก

Reactive: เขียนโปรแกรมแบบ asynchronous ที่ตอบสนองต่อ data stream โดยไม่บล็อก thread — thread ไม่ต้องรอผลแต่ไปทำงานอื่นก่อน เมื่อข้อมูลพร้อมค่อยส่งต่อผ่าน pipeline (Publisher → Operator → Subscriber) ทำให้ใช้ thread จำนวนน้อยรับ request ได้มาก (WebFlux รันบน Reactor Netty แบบ event-loop)

	Blocking	Reactive
Thread	รอจนกว่าจะได้ผล	ไม่รอ ไปทำงานอื่น
Scalability	1 thread per request	few threads, many requests
Return type	Product	Mono<Product>
Server (Spring)	Spring MVC + Tomcat	Spring WebFlux + Reactor Netty
เหมาะกับ	CRUD ทั่วไป	High-concurrency, streaming

```
// Blocking: thread หยุดรอจนกว่า DB/HTTP จะตอบ
Product p = repo.findById("1");           // คืบ Product ตรง ๆ
System.out.println(p.getName());
```

```
// Reactive: ได้ "blueprint" กลับมาทันที ไม่รอ
Mono<Product> m = repo.findById("1");      // ยังไม่มีอะไรทำงาน
m.subscribe(p -> System.out.println(p.getName())); // เริ่มทำงานตอน subscribe
```

2. Mono vs Flux ใช้กรณีไหน และทำไม

	Mono<T>	Flux<T>
จำนวนค่า	0 หรือ 1	0 ถึง N
ใช้เมื่อ	ผลลัพธ์เดียว เช่น findById, save, delete	หลายผลลัพธ์/stream เช่น findAll, search
สร้างด้วย	Mono.just(), Mono.empty()	Flux.just(), Flux.fromIterable(), Flux.range()
ไม่มีผลลัพธ์	Mono<Void> (side-effect เช่น delete)	—

type บอก "จำนวนผลลัพธ์" ตั้งแต่ตอนออกแบบ compiler ช่วยตรวจ และ operator เหมาะกับแต่ละแบบ (เช่น collectList() รวม Flux เป็น Mono<List<T>>, switchIfEmpty/defaultIfEmpty ใช้จัดการกรณี Mono ว่าง) การคืน Flux สำหรับ findById หรือ Mono สำหรับ findAll จะผิดความหมายและเสียข้อมูล

Method ใน Lab	Return	เหตุผล
findById / getById / getProductById	Mono<Product>	id ขำไม่ได้ ได้ผลลัพธ์ 0-1 รายการ
findAll / getAll / getCategory	Flux<Product>	ได้หลายรายการ
save / createProduct	Mono<Product>	บันทึกทีละ 1 รายการ คืนตัวที่บันทึก
delete / deleteById	Mono<Void>	ไม่มีค่าคืน แจ้งแค่สำเร็จ/error
getDiscountedPrice	Mono<Double>	ราคาเดียวของสินค้า 1 ชิ้น

3. WebClient — เรียก HTTP API แบบ non-blocking

WebClient คือ HTTP client ของ Spring WebFlux ที่คืนผลเป็น Mono/Flux แทนการรอ response ตรง ๆ (ต่างจาก RestTemplate ที่ blocking)

```

client.get()           // 1. เลือก HTTP method
    .uri("/products/{id}", "1") // 2. กำหนด path (+ URI variable)
    .retrieve()         // 3. ส่ง request / เริ่มรับ response
    .bodyToMono(Product.class); // 4. แปลง body เป็น Mono<Product>

```

ส่วนของ chain	ทำอะไร
.get() / .post() / .delete()	เลือก HTTP method ของ request
.uri("/products/{id}", id)	กำหนด path (ต่อจาก base URL) และแทนค่า URI variable
.bodyValue(obj)	(POST) ใส่ request body ระบบแปลง object เป็น JSON ให้
.retrieve()	ส่ง request และเริ่มรับ response (ถ้า 4xx/5xx จะเกิด WebClientResponseException)
.bodyToMono(T.class)	แปลง response body เป็น Mono<T> เมื่อคาดหวังผลลัพธ์เดียว (หรือ Void ถ้าไม่มี body)
.bodyToFlux(T.class)	แปลง response body เป็น Flux<T> เมื่อคาดหวังหลายผลลัพธ์ (JSON array)
.doOnNext(fn)	(เสริม) ทำ side-effect เช่น log เมื่อได้ค่า โดยไม่เปลี่ยนค่า

หมายเหตุ: เหมือน Mono/Flux ทั่วไป request จริงจะถูกยิงเมื่อมีการ subscribe() เท่านั้น

4. Code พร้อมคำอธิบาย

4.1 ProductRepository (in-memory)

```

public class ProductRepository {

    // in-memory storage: key = id, value = Product (thread-safe)
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // (constructor seed data ของเดิม + ใส่รหัสนักศึกษาในชื่อ product id=1)

```

```

// 1. หา 1 รายการ -> Mono<Product> (0 หรือ 1 ค่า)
public Mono<Product> findById(String id) {
    Product product = store.get(id); // get() ได้ null ถ้าไม่พบ
    // ไม่พบ -> Mono.empty(), พบ -> Mono.just(product)
    return product != null ? Mono.just(product) : Mono.empty();
}

// 2. หาทั้งหมด -> Flux<Product> (0..N ค่า)
public Flux<Product> findAll() {
    // แปลง Collection เป็น Flux ส่งทีละ element
    return Flux.fromIterable(store.values());
}

// 3. บันทึก -> คืน product ที่บันทึกแล้ว
public Mono<Product> save(Product product) {
    store.put(product.getId(), product); // เขียนลง map
    return Mono.just(product);
}

// 4. ลบ -> Mono<Void> (ไม่มีผลลัพธ์ มีแค่ side-effect)
public Mono<Void> deleteById(String id) {
    store.remove(id); // ลบออกจาก map
    return Mono.empty(); // ไม่มีค่าคืน = Mono<Void>
}

// 5. กรองตาม category -> Flux<Product>
public Flux<Product> findByCategory(String category) {
    return findAll()
        .filter(p -> p.getCategory().equalsIgnoreCase(category)); // ไม่สนใจตัวพิมพ์
}
}

```

4.2 ProductService (business logic)

```

@Service
public class ProductService {

    private final ProductRepository repository;

    // Constructor Injection (DIP): ฟังพา abstraction ที่ Spring ฉีดให้
    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    // 1. ดึง 1 รายการ: ถ้าไม่พบ -> ส่ง error signal
    public Mono<Product> getById(String id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(
                new RuntimeException("Product not found: " + id)));
    }

    // 2. ดึงทั้งหมด
    public Flux<Product> getAll() {
        return repository.findAll();
    }

    // 3. บันทึก: ถ้าไม่มี id ให้ generate ใหม่ก่อน
    public Mono<Product> save(Product product) {
        if (product.getId() == null) {

```

```

        product.setId(java.util.UUID.randomUUID().toString());
    }
    return repository.save(product);
}

// 4. ลบ
public Mono<Void> delete(String id) {
    return repository.deleteByid(id);
}

// 5. กรองตาม category
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// 6. ราคาหลังส่วนลด: หา Product ก่อน แล้ว map เป็นราคา (sync transform)
public Mono<Double> getDiscountedPrice(String id) {
    return repository.getbyid(id).map(p -> p.getDiscountedPrice());
}
}

```

4.3 ProductController (REST endpoints)

```

@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    // GET /products/{id} -> Mono<Product> (ตัวอย่างเดิม)
    @GetMapping("/{id}")
    public Mono<Product> getbyid(@PathVariable String id) {
        return service.getbyid(id);
    }

    // GET /products -> Flux<Product> ; WebFlux subscribe ให้เอง
    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }

    // POST /products ; @RequestBody แปลง JSON -> Product
    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }

    // DELETE /products/{id} -> Mono<Void> (ตอบ 200 body ว่าง)
    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) {
        return service.delete(id);
    }

    // GET /products/category/{category} -> Flux<Product>
    @GetMapping("/category/{category}")

```

```

    public Flux<Product> getByCategory(@PathVariable String category) {
        return service.getByCategory(category);
    }

    // GET /products/{id}/price -> Mono<Double>
    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(@PathVariable String id) {
        return service.getDiscountedPrice(id);
    }
}

```

4.4 ProductWebClient

```

@Component
public class ProductWebClient {

    // base URL ที่ไปที่ server ตัวเอง
    private final WebClient client = WebClient.create("http://localhost:8080");

    // GET 1 รายการ (ตัวอย่างเดิม)
    public Mono<Product> getProductById(String id) {
        return client.get()
            .uri("/products/{id}", id)
            .retrieve()
            .bodyToMono(Product.class);
    }

    // GET ทั้งหมด -> Flux
    public Flux<Product> getAllProducts() {
        return client.get()
            .uri("/products")
            .retrieve()
            .bodyToFlux(Product.class);
    }

    // POST: bodyValue() ใส่ request body (serialize เป็น JSON)
    public Mono<Product> createProduct(Product product) {
        return client.post()
            .uri("/products")
            .bodyValue(product)
            .retrieve()
            .bodyToMono(Product.class);
    }

    // DELETE: response ไม่มี body -> bodyToMono(Void.class)
    public Mono<Void> deleteProduct(String id) {
        return client.delete()
            .uri("/products/{id}", id)
            .retrieve()
            .bodyToMono(Void.class);
    }

    // GET ตาม category -> Flux
    public Flux<Product> getByCategory(String category) {
        return client.get()
            .uri("/products/category/{category}", category)
            .retrieve()
            .bodyToFlux(Product.class);
    }
}

```

```
// GET ราคาหลังลด + doOnNext (side-effect: log ตอนได้ค่า ไม่เปลี่ยนค่า)
public Mono<Double> getDiscountedPrice(String id) {
return client.get()
    .uri("/products/{id}/price", id)
    .retrieve()
    .bodyToMono(Double.class)
    .doOnNext(price -> System.out.println("Price: " + price));
}
}
```

5. ผลการทดสอบ (Screenshot)

รันด้วย `mvn spring-boot:run` ทดสอบผ่าน Browser หรือ Postman

1. GET `http://localhost:8080/products` Flux<Product>

Postman interface showing a GET request to `http://localhost:8080/products`. The response is a 200 OK status with a JSON array of 3 products.

id	name	category	brand	stock	price	discountType	discountedPrice
0 1	iPhone 15 Pro (673380287-8 SEC 1)	Electronics	Apple	50	39900	MEMBER	35910
1 2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2 3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

2. GET `http://localhost:8080/products/1` Mono<Product>

Postman interface showing a GET request to `http://localhost:8080/products/1`. The response is a 200 OK status with a JSON object representing the first product.

id	1
name	iPhone 15 Pro (673380287-8 SEC 1)
category	Electronics
brand	Apple
stock	50
price	39900
discountType	MEMBER
discountedPrice	35910

3. POST http://localhost:8080/products Mono<Product>

COLLECTIONS

New Collection

POST Add

DEL Delete

GET Products

GET Electronics

GET price

GET Product 1

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

FLAMES

Get startedNew CollecPOST AddDEL DeleteGET ProducGET Unentitled RGET ElectronGET priceGET http://loGET Produc+

New Collection > Add

POSThttp://localhost:8080/products

DocsParamsAuthorizationHeaders 8BodyScriptsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

1{"name":"Test","category":"Electronics","brand":"X","stock":999,"price":99999,"discountType":"NONE"}

BodyCookiesHeaders 2Test Results200 OK8 ms246 B

{ }JSONPreviewVisualize

id	aa5d2bc4-26a2-4cbd-b535-0130db253218
name	Test
category	Electronics
brand	X
stock	999
price	99999
discountType	NONE
discountedPrice	99999

COLLECTIONS

New Collection

POST Add

DEL Delete

GET Products

GET Electronics

GET price

GET Product 1

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

FLAMES

Get startedNew CollecPOST AddDEL DeleteGET ProducGET Unentitled RGET ElectronGET priceGET http://loGET Produc+

New Collection > Products

GETlocalhost:8080/productsSend

DocsParamsAuthorizationHeaders 6BodyScriptsSettings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

BodyCookiesHeaders 2Test Results200 OK4 ms739 BSave Response

{ }JSONPreviewVisualize

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (673380287-8 SEC 1)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920
3	aa5d2bc4-26a2-4cbd-b535-0130db253218	Test	Electronics	X	999	99999	NONE	99999

4. DELETE http://localhost:8080/products/{id} Mono<Void>

COLLECTIONS

New Collection

POST Add

DEL Delete

GET Products

GET Electronics

GET price

GET Product 1

Get started

New Collec

POST Add

DEL Deletc

GET Produc

GET Untitled R

GET Electron

GET price

GET http://lo

GET Produc

+

▼

New Collection > Delete

DELETE

http://localhost:8080/products/aa5d2bc4-26a2-4cbd-b535-0130db253218

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

This request does not have a body

Body

Cookies

Headers 1

Test Results

200 OK

2 ms

40 B

Raw

Preview

Visualize

▼

1

COLLECTIONS

New Collection

POST Add

DEL Delete

GET Products

GET Electronics

GET price

GET Product 1

Get started

New Collec

POST Add

DEL Deletc

GET Produc

GET Untitled R

GET Electron

GET price

GET http://lo

GET Produc

+

▼

No environment

▼

New Collection > Products

GET

localhost:8080/products

Save

Share

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

Cookies

Headers 2

Test Results

200 OK

7 ms

566 B

Save Response

{}

JSON

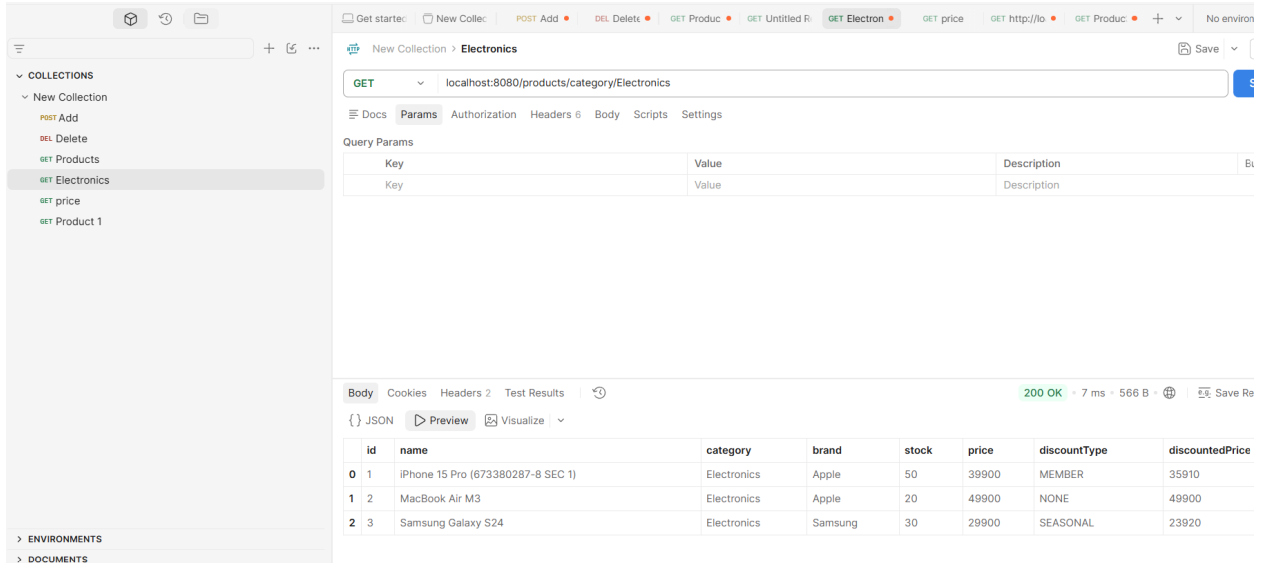
Preview

Visualize

▼

id	name	category	brand	stock	price	discountType	discountedPrice
0 1	iPhone 15 Pro (673380287-8 SEC 1)	Electronics	Apple	50	39900	MEMBER	35910
1 2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2 3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

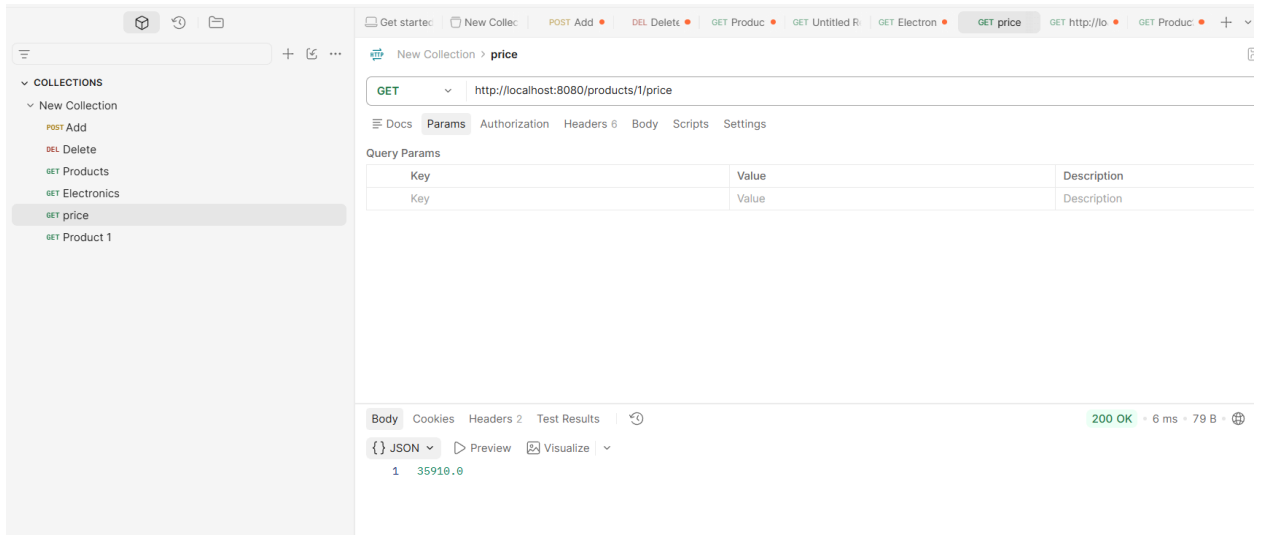
5. GET `http://localhost:8080/products/category/Electronics` Flux<Product>



Postman interface showing a GET request to `localhost:8080/products/category/Electronics`. The response is a 200 OK status with a 7 ms response time and 566 B of data. The response body is a JSON array of product objects.

id	name	category	brand	stock	price	discountType	discountedPrice
0 1	iPhone 15 Pro (673380287-8 SEC 1)	Electronics	Apple	50	39900	MEMBER	35910
1 2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2 3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

6. GET `http://localhost:8080/products/1/price` Mono<Double>



Postman interface showing a GET request to `http://localhost:8080/products/1/price`. The response is a 200 OK status with a 6 ms response time and 79 B of data. The response body is a JSON object containing the price.

```
{
  "price": 35910.0
}
```